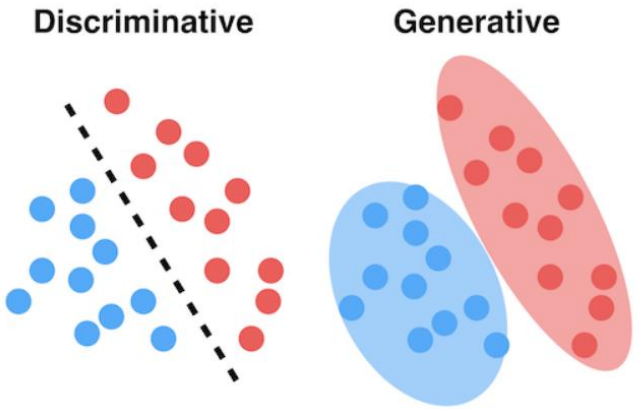
# Chapter 10
# Generative Models

Dr. Aryaf AL-Adwan

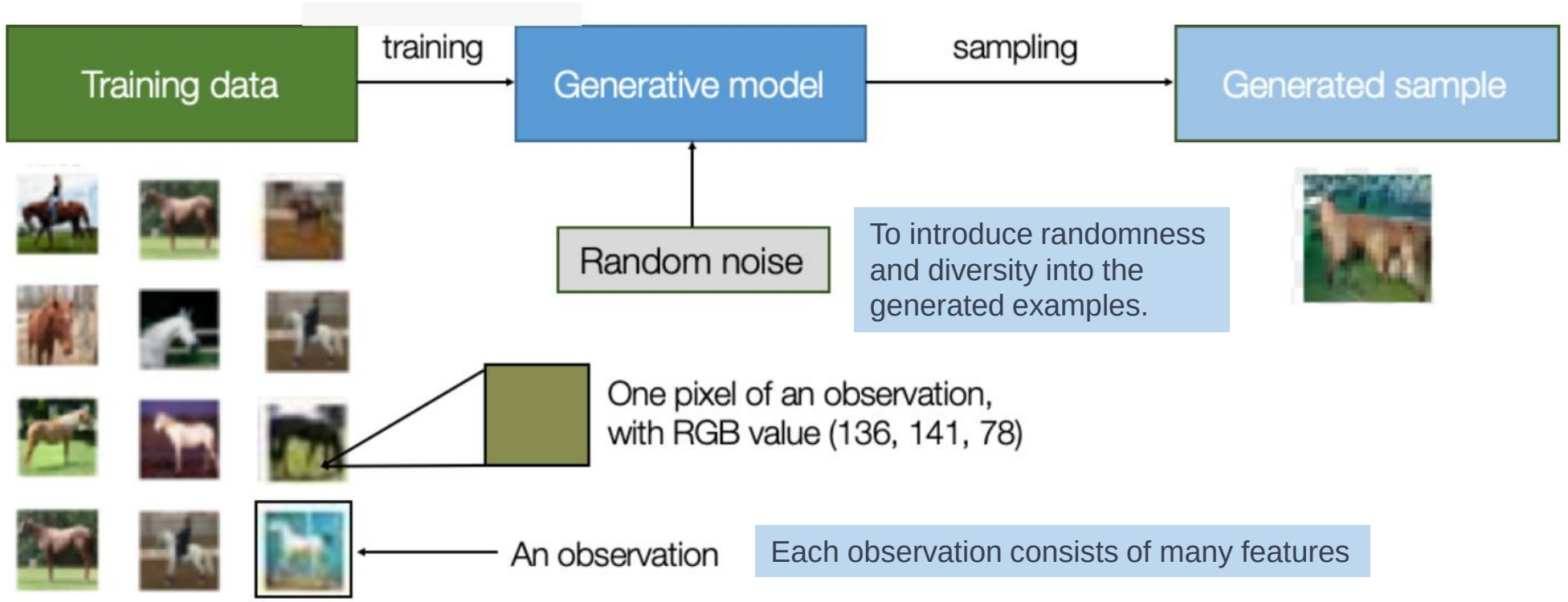
# Discriminative and Generative Models

- Machine learning classification models can be classified into two types of models
- **Discriminative model :** draw boundaries between classes in the data space
- **Generative model :** A generative model is a probabilistic model that describes how a dataset is generated. By learning the underlying structure of the data, the model is able to generate new examples that are similar to those in the training set.



# Generative Models

Suppose we have a dataset containing images of horses. We may wish to build a model that can **generate** a new image of a horse that has never existed but still looks real because the model has learned the general rules that govern the appearance of a horse.



# Cont.

- A generative model must be **probabilistic** rather than deterministic.
- If our model is merely a fixed calculation, such as taking the average value of each pixel in the dataset, it is not generative because the model produces the same output every time.
- Generative models are generally considered to be unsupervised learning models.

*Discriminative modeling* estimates p ( y | x ) —the probability of a label *y* given observation **x**.

*Generative modeling* estimates p ( x ) —the probability of observing observation **x**.

Discriminative modeling attempts to estimate the probability that an observation x belongs to category y. Generative modeling doesn't care about labeling observations. Instead, it attempts to estimate the probability of seeing the observation.

# Cont.

❖ **Generative models**

1. Bayesian network (e.g. Naive Bayes)
2. Gaussian mixture model
3. Hidden Markov model
4. Variational Autoencoders (VAEs)
5. Generative Adversarial Networks (GANs)

❖ **Discriminative models**

1. k-nearest neighbors algorithm
2. Logistic regression
3. Support Vector Machines
4. Decision Tree Learning
5. Random Forest

# Naive Bayes

- The Naive Bayes algorithm is a **generative** model because it learns the probabilities of different features occurring in the training data, and then uses these probabilities to generate a model that can classify new examples.
- Used for classification tasks.
- Naive Bayes is often used for text classification tasks, such as spam detection or sentiment analysis. It is also used in other areas, such as medical diagnosis and credit fraud detection.

# Why is it called Naive Bayes?

- The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:

1. **Naive**: It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
2. **Bayes**: It is called Bayes because it depends on the principle of Bayes' Theorem.

# Bayes' Theorem

- Bayes' theorem is also known as Bayes' Rule or Bayes' law, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- **Where,**
- **P(A|B) is Posterior probability**: Probability of hypothesis A on the observed event B.
- **P(B|A) is Likelihood probability**: Probability of the evidence given that the probability of a hypothesis is true.
- **P(A) is Prior Probability:** Probability of hypothesis before observing the evidence.
- **P(B) is Marginal Probability:** Probability of Evidence.

# Math behind Naive Bays Algorithm

- Given a features vector X=(x1,x2,…,xn) and a class variable y, Bayes Theorem states that:

$$P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$

- For all the features in vector X

$$P(y|X) = \frac{P(x_1|y)*P(x_2|y) \ldots\ldots P(x_n|y) * P(y)}{P(x_1) * P(x_2) \ldots . P(x_n)}$$

# Cont.

- And as denominator remains constant for all values, the posterior probability can then be:

$$P(y|x_1, x_2, \ldots, x_n) \propto P(y) \prod_{i=1}^{n} P(x_i|y)$$

- One common rule is to pick the hypothesis that's most probable; this is known as the maximum a posteriori or MAP decision rule.

$$y = argmax_y P(y) \prod_{i=1}^{n} P(x_i|y)$$

To make the prediction, we would select the class label that maximizes the probability, like this:

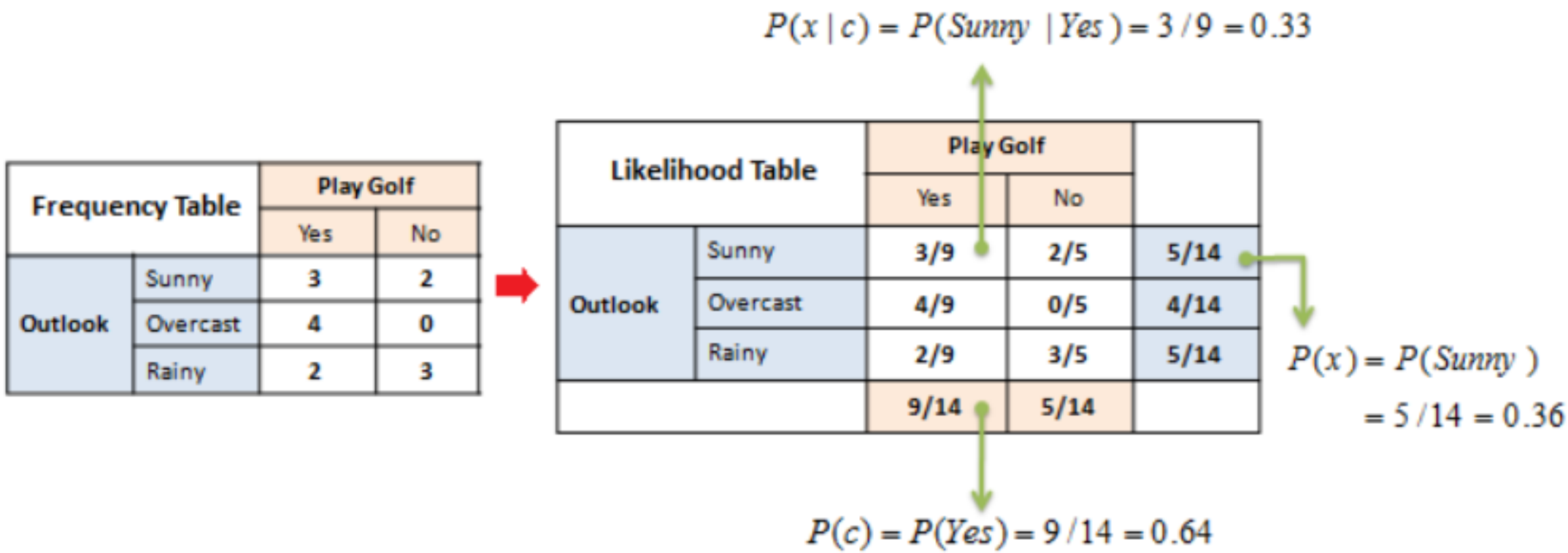
prediction = arg max(P(A|X), P(B|X))

If P(A|X) > P(B|X), the prediction would be A. If P(B|X) > P(A|X), the prediction would be B.

# Example

- Weather dataset

| Outlook | Temp | Humidity | Windy | Play Golf |
|---|---|---|---|---|
| Rainy | Hot | High | False | No |
| Rainy | Hot | High | True | No |
| Overcast | Hot | High | False | Yes |
| Sunny | Mild | High | False | Yes |
| Sunny | Cool | Normal | False | Yes |
| Sunny | Cool | Normal | True | No |
| Overcast | Cool | Normal | True | Yes |
| Rainy | Mild | High | False | No |
| Rainy | Cool | Normal | False | Yes |
| Sunny | Mild | Normal | False | Yes |
| Rainy | Mild | Normal | True | Yes |
| Overcast | Mild | High | True | Yes |
| Overcast | Hot | Normal | False | Yes |
| Sunny | Mild | High | True | No |




Posterior Probability: $P(c \mid x) = P(Yes \mid Sunny) = 0.33 \times 0.64 \div 0.36 = 0.60$

| Frequency Table | | Play Golf | |
|---|---|---|---|
| | | Yes | No |
| Outlook | Sunny | 3 | 2 |
| | Overcast | 4 | 0 |
| | Rainy | 2 | 3 |

$$P(x \mid c) = P(Sunny \mid No) = 2/5 = 0.4$$

| | | Play Golf | | |
|---|---|---|---|---|
| | | Yes | No | |
| Outlook | Sunny | 3 | 2 | 5 |
| | Overcast | 4 | 0 | 4 |
| | Rainy | 2 | 3 | 5 |
| | | 9 | 5 | 14 |

$$P(x) = P(Sunny) = 5/14 = 0.36$$

$$P(c) = P(No) = 5/14 = 0.36$$

Posterior Probability:

$$P(c \mid x) = P(No \mid Sunny) = 0.40 \times 0.36 \div 0.36 = 0.40$$

*Example 2*:

In this example we have 4 inputs (predictors). The final posterior probabilities can be standardized between 0 and 1.

| Outlook | Temp | Humidity | Windy | Play |
|---|---|---|---|---|
| Rainy | Cool | High | True | ? |

$$P(Yes \mid X) = P(Rainy \mid Yes) \times P(Cool \mid Yes) \times P(High \mid Yes) \times P(True \mid Yes) \times P(Yes)$$

$$P(Yes \mid X) = 2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14 = 0.00529$$

$$P(No \mid X) = P(Rainy \mid No) \times P(Cool \mid No) \times P(High \mid No) \times P(True \mid No) \times P(No)$$

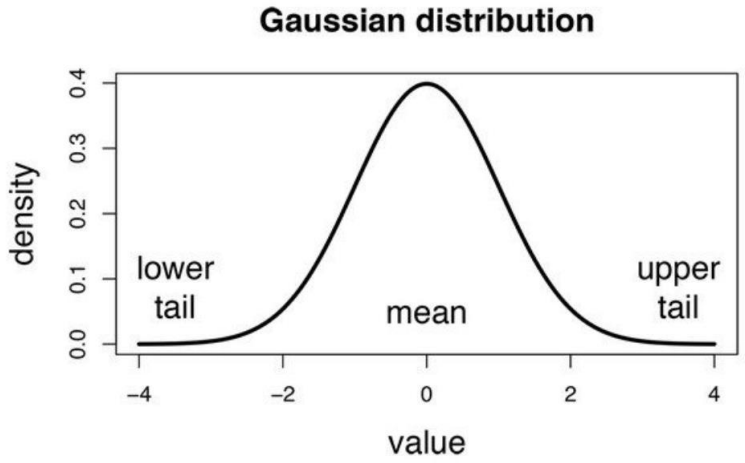
$$P(No \mid X) = 3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14 = 0.02057$$

# Types of Naive Bayes algorithm

- **Gaussian**: The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial**: The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc.
- **Bernoulli**: The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

# Gaussian Naïve Bayes

- So far we've seen the computations when the X's are categorical.
- But how to compute the probabilities when X is a continuous variable?
- each feature are assumed to be distributed according to a Gaussian distribution (Normal distribution).




$$p(x_i|y_j) = \frac{1}{\sqrt{2\pi\sigma_j^2}} e^{-\frac{(x_i-\mu_j)^2}{2\sigma_j^2}}$$

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Mean

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Normal distribution

$$\sigma = \left[\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \mu)^2\right]^{0.5}$$

Standard deviation

| | | Humidity | *Mean* | *StDev* |
|---|---|---|---|---|
| **Play** | yes | 86 96 80 65 70 80 70 90 75 | 79.1 | 10.2 |
| **Golf** | no | 85 90 70 95 91 | 86.2 | 9.7 |

$$P(\text{humidity} = 74 \mid \text{play} = \text{yes}) = \frac{1}{\sqrt{2\pi}(10.2)} e^{-\frac{(74-79.1)^2}{2(10.2)^2}} = 0.0344$$

$$P(\text{humidity} = 74 \mid \text{play} = \text{no}) = \frac{1}{\sqrt{2\pi}(9.7)} e^{-\frac{(74-86.2)^2}{2(9.7)^2}} = 0.0187$$

# Generative Adversarial Network or GAN

A Generative Adversarial Network or GAN is defined as the technique of generative modeling used to generate new data sets based on training data sets. The newly generated data set appears similar to the training data sets.

# Generative adversarial networks

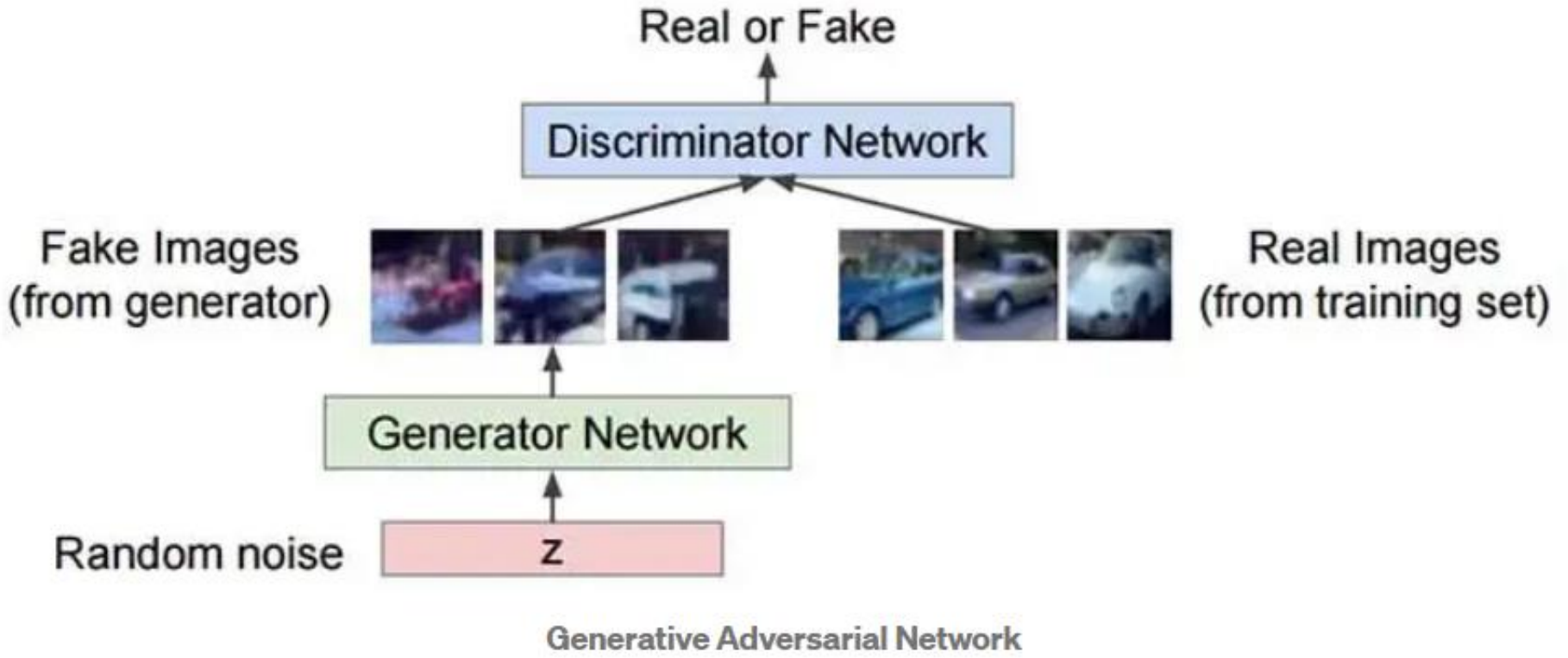
- Are the most popular and recent **unsupervised machine learning** innovations developed by Ian J. Goodfellow in 2014.
- Generative adversarial networks (GANs) are a type of generative model that uses random noise as input to generate new examples. GANs consist of **two neural networks**:
  1. **generator network:** takes as input a random noise vector and generates samples that are similar to a set of training data (such as images, text, etc.).
  2. **discriminator network**: to determine whether a sample is real (from the training data) or fake (generated by the generator).
  - The two networks are trained together, with the generator trying to produce samples that the discriminator will classify as real, and the discriminator trying to correctly identify the generated samples as fake. This creates an adversarial relationship between the two networks, as the generator is trying to fool the discriminator, while the discriminator is trying to correctly identify the generated samples.
  - It is most often used in various ML applications, such as image generation, video generation, and speech generation.

# Why is it called Generative Adversarial Network ?

- Generative Adversarial Network consists of three words, and each of them has its separate meaning, which is as follows:

1. **Generative:** It is used to learn a generative model that visually explains how data is generated.
2. **Adversarial:** As both neural networks compete with each other or are adversarial to one another, hence training of the model is done in an adversarial manner.
3. **Networks:** It uses deep neural networks to train models, hence called networks.

# GAN = Generator + Discriminative




Generative Adversarial Network

The end goal of training a GAN is to train the generator to produce samples that are indistinguishable from real samples. At the end of the training, the generator should be able to produce samples that are similar to the training data. This is achieved by the generator and discriminator training each other until the discriminator is unable to distinguish the generated samples from real samples, meaning the generator is producing samples that look like real samples.

Once this point is reached, the generator can be used to generate new samples that are similar to the training data, for example if the data is images of faces, the generator can be used to generate new and realistic images of faces.

# The math behind the GANs

## Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

**Generator network**: try to fool the discriminator by generating real-looking images
**Discriminator network**: try to distinguish between real and fake images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[ \mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data G(z)

- Discriminator ($\theta_d$) wants to **maximize objective** such that D(x) is close to 1 (real) and D(G(z)) is close to 0 (fake)
- Generator ($\theta_g$) wants to **minimize objective** such that D(G(z)) is close to 1 (discriminator is fooled into thinking generated G(z) is real)

# Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[ \mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[ \mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

# GAN Algorithm

**for** number of training iterations **do**

  **for** $k$ steps **do**

- Sample minibatch of $m$ noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of $m$ examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[ \log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

  **end for**

- Sample minibatch of $m$ noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

**end for**

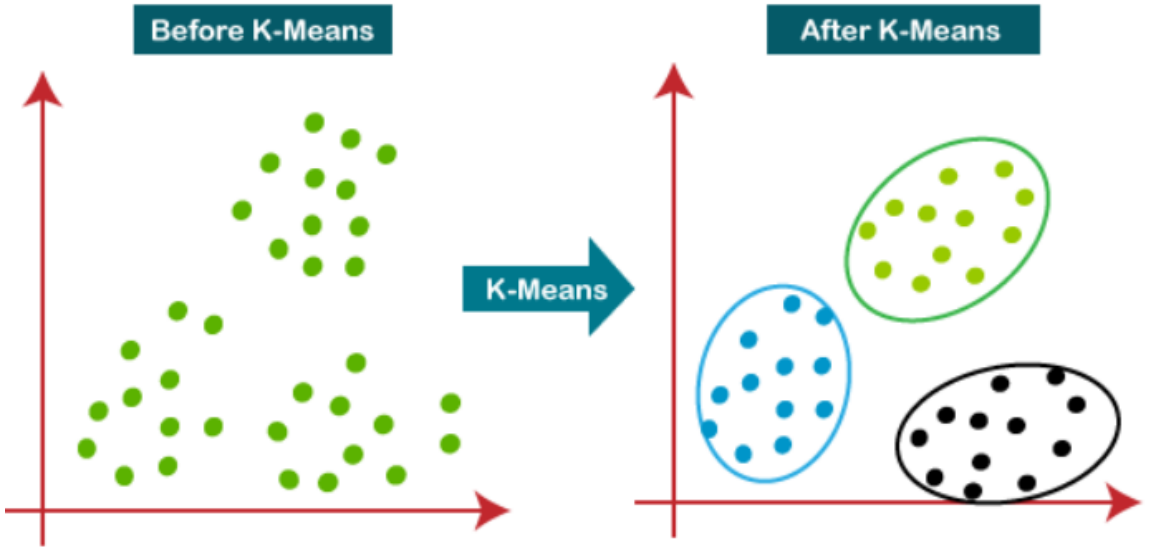
The log function is used in the GAN objective function because it provides a useful mathematical property called the log-likelihood. The log-likelihood is a measure of how well a probability model fits the data. In the case of a GAN, the log-likelihood is used to measure how well the generator's output fits the real data.

# Clustering- Unsupervised Learning

- Clustering is the process of grouping **similar** data points together in a dataset.
- Clustering algorithms are unsupervised machine learning methods that identify patterns or structures in the data without the use of labeled training data.
- There are several types of clustering algorithms, such as hierarchical, density-based, and centroid-based (e.g. K-Means).
- Clustering is used in a wide range of applications, such as image and speech recognition, natural language processing, market segmentation, and anomaly detection.

# K Means Clustering

- K-Means is a clustering algorithm (unsupervised learning) used to group similar data points together. It works by dividing a dataset into a specified number of clusters (K) based on the means (centroids) of the data points in each cluster.



# K Means Algorithm

**Step-1:** Select the number K to decide the number of clusters.

**Step-2:** Randomly initialize 'K' centroids as each cluster will have one center. So, for example, if we have 7 clusters, then we would initialize seven centroids.

**Step-3:** Compute the Euclidian distance of each current data point to all the cluster centers.

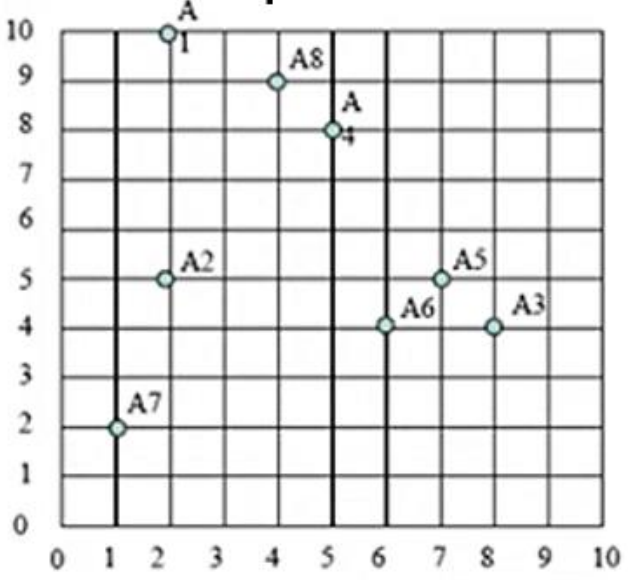
Let us assume we have two points, A1(X1, Y1) and B2(X2, Y2). Then the Euclidian distance between the two points would be the following:-

$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2-X_1)^2+(Y_2-Y_1)^2}$$

**Step-4:** update the cluster center locations by taking the mean of the data points assigned.

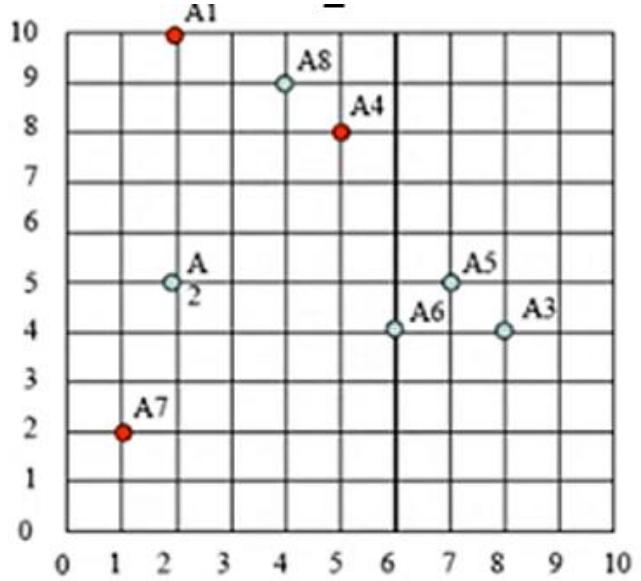
**Step-5:** Repeat the above two steps until convergence.

# Example



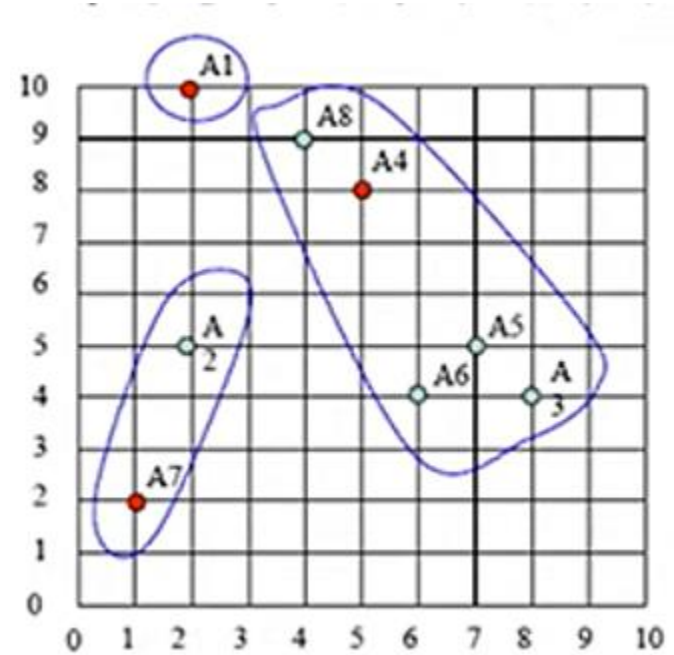

Cluster the following eight points (with (x, y) representing locations) into three clusters A1(2, 10) A2(2, 5) A3(8, 4) A4(5, 8) A5(7, 5) A6(6, 4) A7(1, 2) A8(4, 9).

Initial cluster centers are: A1(2, 10), A4(5, 8) and A7(1, 2).



Iteration 1

| | | (2, 10) | (5, 8) | (1, 2) | |
|---|---|---|---|---|---|
| | **Point** | **Dist Mean 1** | **Dist Mean 2** | **Dist Mean 3** | **Cluster** |
| A1 | (2, 10) | 0 | 5 | 9 | 1 |
| A2 | (2, 5) | 5 | 6 | 4 | 3 |
| A3 | (8, 4) | 12 | 7 | 9 | 2 |
| A4 | (5, 8) | 5 | 0 | 10 | 2 |
| A5 | (7, 5) | 10 | 5 | 9 | 2 |
| A6 | (6, 4) | 10 | 5 | 7 | 2 |
| A7 | (1, 2) | 9 | 10 | 0 | 3 |
| A8 | (4, 9) | 3 | 2 | 10 | 2 |




**Cluster 1**
(2, 10)

**Cluster 2**
(8, 4)
(5, 8)
(7, 5)
(6, 4)
(4, 9)

**Cluster 3**
(2, 5)
(1, 2)

| point | mean1 |
|---|---|
| *x1*, *y1* | *x2*, *y2* |
| (2, 10) | (2, 10) |

$$\rho(a, b) = |x2 - x1| + |y2 - y1|$$

$$\begin{aligned}\rho(point, mean1) &= |x2 - x1| + |y2 - y1| \\ &= |2 - 2| + |10 - 10| \\ &= 0 + 0 \\ &= 0\end{aligned}$$

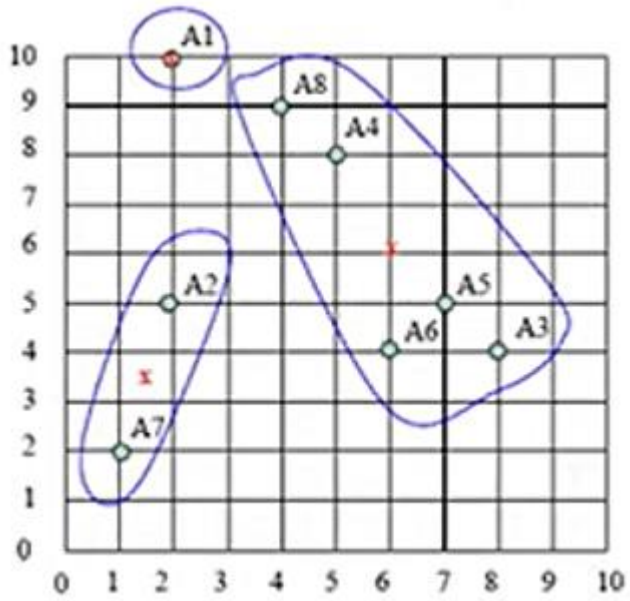
| point | mean2 |
|---|---|
| *x1*, *y1* | *x2*, *y2* |
| (2, 10) | (5, 8) |

$$\rho(a, b) = |x2 - x1| + |y2 - y1|$$

$$\begin{aligned}\rho(point, mean2) &= |x2 - x1| + |y2 - y1| \\ &= |5 - 2| + |8 - 10| \\ &= 3 + 2 \\ &= 5\end{aligned}$$

# Recompute the new clusters centers

| Cluster 1 | Cluster 2 | Cluster 3 |
|---|---|---|
| (2, 10) | (8, 4) | (2, 5) |
| | (5, 8) | (1, 2) |
| | (7, 5) | |
| | (6, 4) | |
| | (4, 9) | |




For Cluster 1, we only have one point A1(2, 10), which was the old mean, so the cluster center remains the same.

For Cluster 2, we have ( (8+5+7+6+4)/5, (4+8+5+4+9)/5 ) = (6, 6)

For Cluster 3, we have ( (2+1)/2, (5+2)/2 ) = (1.5, 3.5)

# Calculate the distances

| | | (2, 10) | (6, 6) | (1.5, 3.5) | |
|---|---|---|---|---|---|
| | Point | Dist Mean 1 | Dist Mean 2 | Dist Mean 3 | **Cluster** |
| A1 | (2, 10) | 0 | 8 | 7 | 1 |
| A2 | (2, 5) | 5 | 5 | 2 | 3 |
| A3 | (8, 4) | 12 | 4 | 7 | 2 |
| A4 | (5, 8) | 5 | 3 | 8 | 2 |
| A5 | (7, 5) | 10 | 2 | 7 | 2 |
| A6 | (6, 4) | 10 | 2 | 5 | 2 |
| A7 | (1, 2) | 9 | 9 | 2 | 3 |
| A8 | (4, 9) | 3 | 5 | 8 | 1 |

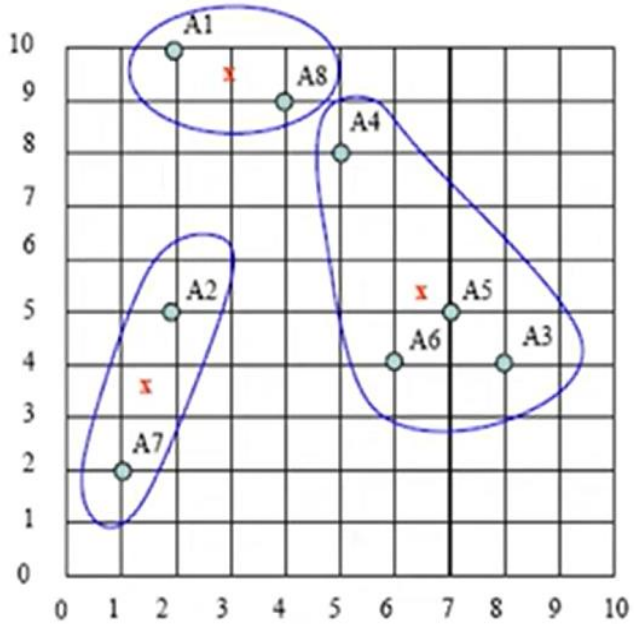
| Cluster 1 | Cluster 2 | Cluster3 |
|---|---|---|
| (2,10) | (8,4) | (2,5) |
| (4,9) | (5,8) | (1,2) |
| | (7,5) | |
| | (6,4) | |

# Recompute the new clusters centers

In Cluster 1, we have points 1 and 8. Therefore the centroid is: ((2+4)/2,(10+9)/2)=(3,9.5)

In Cluster 2, we have points 3,4,5 and 6. Therefore, the centroid is: ((8+5+7+6)/4,(4+8+5+4)/4)=(6.5,5.25)

For Cluster 3, we have points 2 and 7. Therefore, the centroid is: ((2+1)/2,(5+2)/2)=(1.5,3.5)



# Find the distances based on the new clusters

| | | (3, 9.5) | (6.5 ,5.25) | (1.5, 3.5) | |
|---|---|---|---|---|---|
| | Point | Dist Mean 1 | Dist Mean 2 | Dist Mean 3 | **Cluster** |
| A1 | (2, 10) | 1.5 | 9.25 | 7 | **1** |
| A2 | (2, 5) | 5.5 | 4.75 | 2 | **3** |
| A3 | (8, 4) | 10.5 | 2.75 | 7 | **2** |
| A4 | (5, 8) | 3.5 | 4.25 | 8 | **1** |
| A5 | (7, 5) | 8.5 | 0.75 | 7 | **2** |
| A6 | (6, 4) | 8.5 | 1.75 | 5 | **2** |
| A7 | (1, 2) | 9.5 | 8.75 | 2 | **3** |
| A8 | (4, 9) | 1.5 | 6.25 | 8 | **1** |

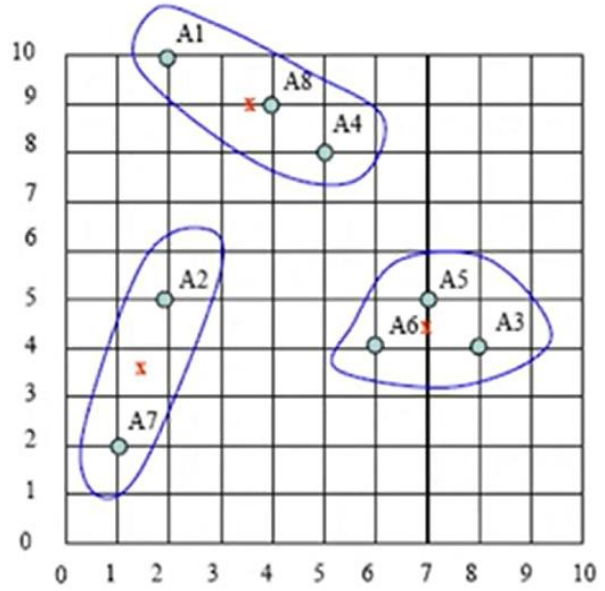
| Cluster 1 | Cluster 2 | Cluster3 |
|---|---|---|
| (2,10) | (8,4) | (2,5) |
| (5,8) | (6,4) | (1,2) |
| (4,9) | (7,5) | |
| | | |

# Recompute the new clusters centers

In Cluster 1, we have points 1, 4, and 8. Therefore the centroid is: $((2+5+4)/2,(10+8+9)/2)=(3.67,9)$

In Cluster 2, we have points 3,5 and 6. Therefore, the centroid is: $((8+7+6)/4,(4+5+4)/4)=(7,4.3)$

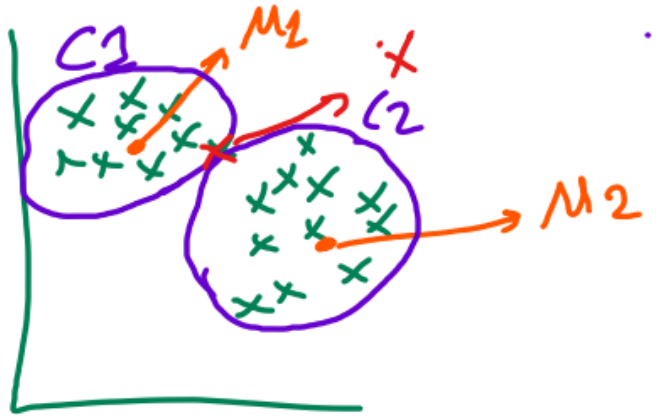
For Cluster 3, we have points 2 and 7. Therefore, the centroid is: $((2+1)/2,(5+2)/2)=(1.5,3.5)$




**Repeat the process until convergence**

# But K-Means is considered as hard assignment clustering !!

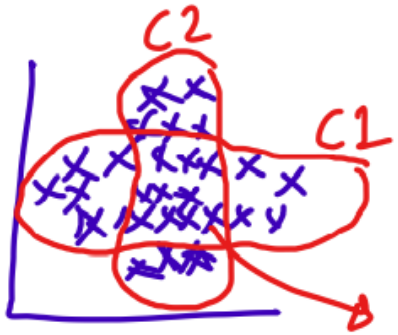
- In hard clustering, each data point is assigned to exactly one cluster and no other clusters.
- In soft clustering or fuzzy clustering each data point is assigned to multiple clusters, with a probability or membership value indicating how likely it belongs to each cluster.




Hard??

$p(x|c_1)$ → x belongs to c1 with specific probability

$p(x|c_2)$ → x belongs to c2 with another probability

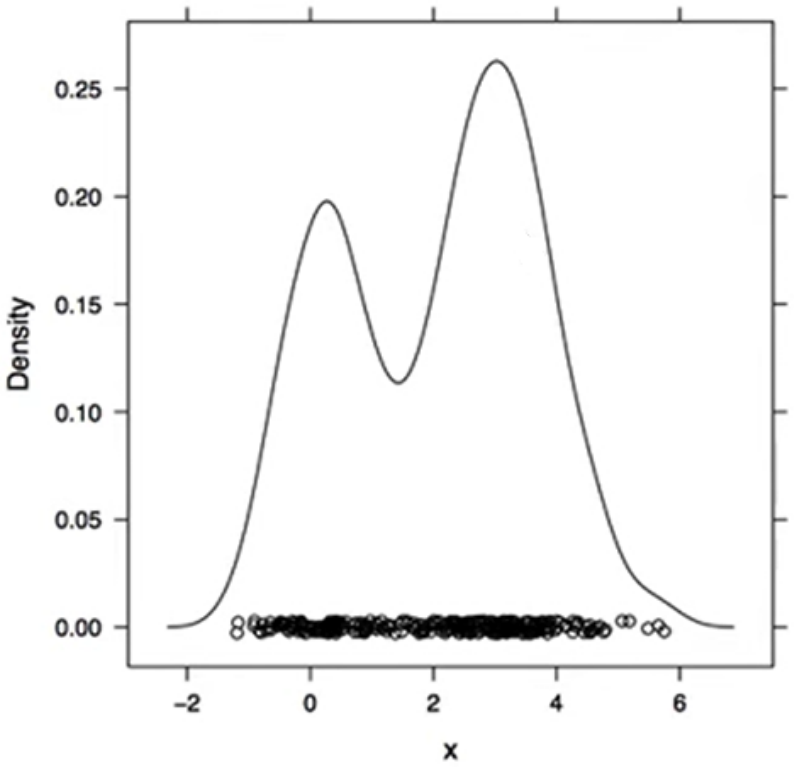


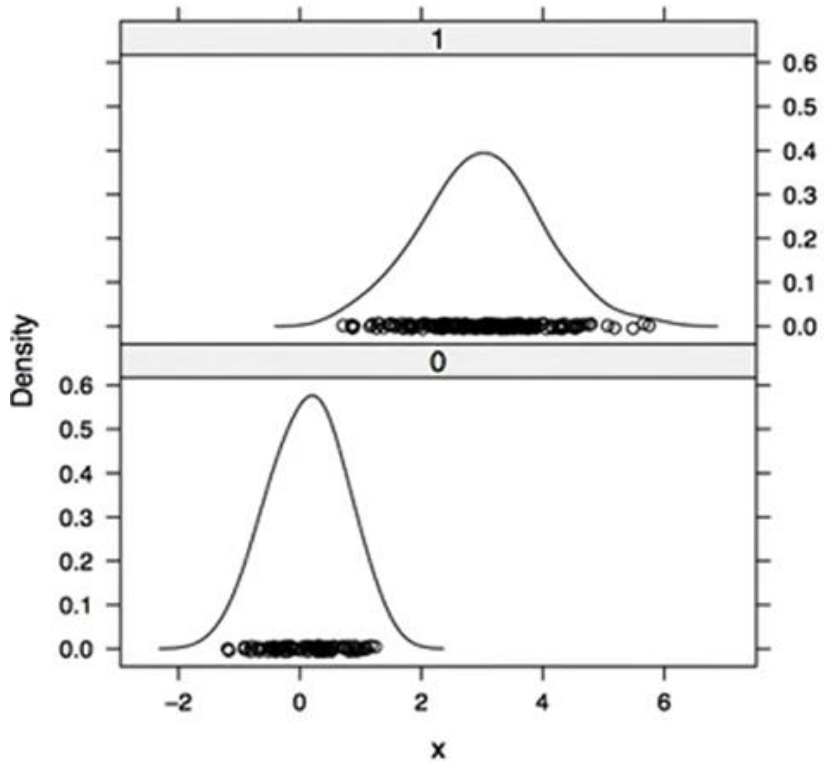

K-Means X

GMM ✓ ⇒ Soft assignment

# Mixture Model

Mixture Model is a type of density model which comprises of a number of component functions, usually Gaussian.



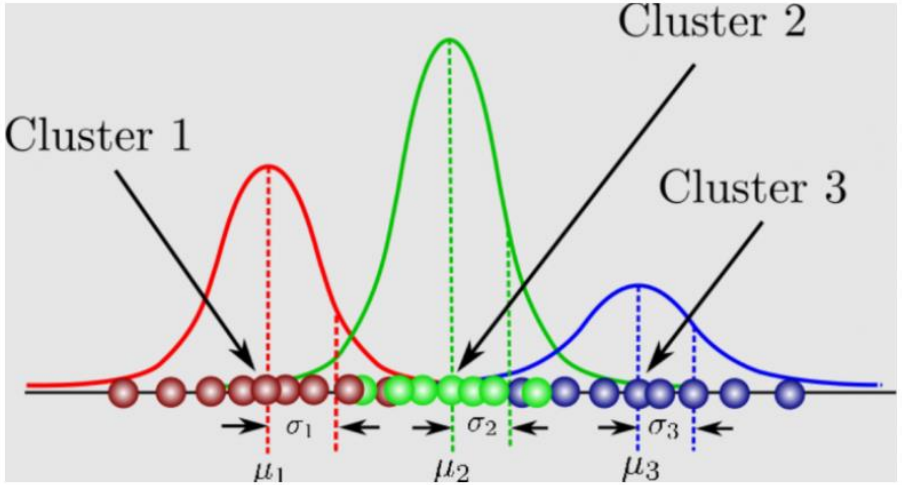




$$\mu_0 = 0, \quad \sigma_0 = .5$$
$$\mu_1 = 3, \quad \sigma_1 = 1$$

We have two distributions, one with a smaller mean and one with a greater mean, and we have observed some points from them. However, we no longer have the ability to determine which observations belong to which distribution.

# Gaussian Mixture Models (GMM)

- They are used for **generative unsupervised** learning or clustering.
- They are also called **Expectation-Maximization Clustering.**
- They are used to classify data into different categories based on the **probability distribution**.
- Additionally, GMMs can be used to estimate the probability that a new data point belongs to each cluster.
- Gaussian mixture models are also relatively **robust to outliers**, meaning that they can still yield accurate results even if there are some data points that do not fit neatly into any of the clusters.
- All these make GMMs a **flexible and powerful** tool for clustering data.



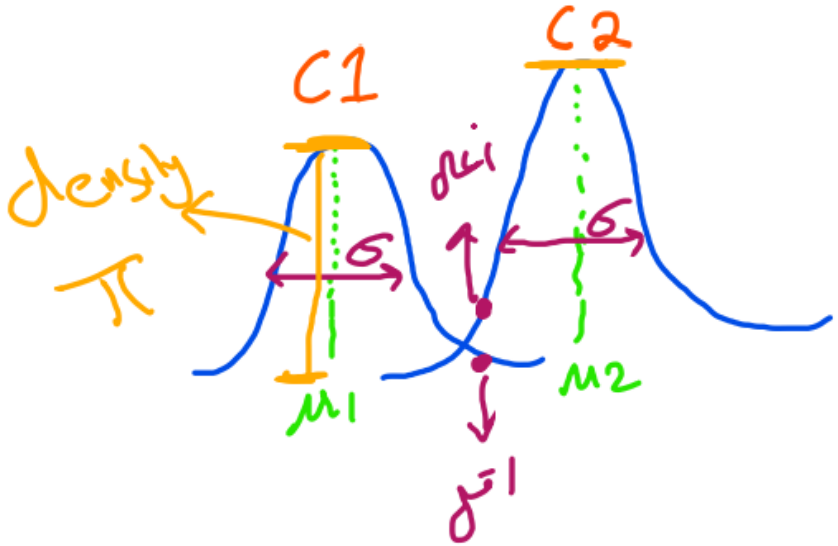
# Expectation-Maximization Algorithmتحقيق أقصى قدر للتوقع

- The EM (Expectation-Maximization) algorithm is one of the most commonly used terms in machine learning to obtain maximum likelihood estimates of variables that are sometimes observable and sometimes not.

- It does this by using the information from the observed data to update the parameters in a way that makes the observed data more likely. It keeps doing this updating process until the parameters are as good as they can be, given the observed data.

# EM algorithm Steps

1. Expectation step (E - step): It involves the estimation (guess) of all missing values in the dataset so that after completing this step, there should not be any missing value.
2. Maximization step (M - step): This step involves the use of estimated data in the E-step and updating the parameters.
3. Repeat E-step and M-step until the convergence of the values occurs.




① $\mu$ ??

② $\Sigma$ $\sigma$ ??

③ $\pi$ ??

} 3 parameters

GMM

$p(y=k) \Rightarrow \pi_k$

$\Rightarrow$ expectation التوقع

$N(x_i | y_k, \mu_k, \Sigma_k, \pi_k) \Rightarrow \pi$ ?? $\mu$

$\Sigma$ ??

EM → expectation
Maximization ⇒ $\pi, \Sigma, M$ الهدف هو ايجاد ال
لدقة عالية

Steps :-

① Initialize $M, \Sigma, \pi$ (Guess)

if i have 3 clusters

c1 → $M_1, \Sigma_1, \pi_1$

c2 → $M_2, \Sigma_2, \pi_2$

c3 → $M_3, \Sigma_3, \pi_3$

② E-Step :-

To find the Posteriem Probability

$$p(x_i \mid y_k) = \frac{\pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_j^K \pi_j \, \mathcal{N}(x_i \mid \mu_j, \Sigma_j)} \longrightarrow$$

(3) M-step: $\mu, \Sigma, \pi \Rightarrow$ update them using

(1) $\mu_k = \dfrac{\sum p(x_i \mid y_k)}{n_k}$

(2) $\Sigma_k = \dfrac{1}{n_k} \sum p(x_i \mid y_k)(x_i - \mu_k)(x_i - \mu_k)^T$

(3) $\pi_k = \dfrac{n_k}{n}$ $\Rightarrow \pi_k, \Sigma_k, \mu_k \rightarrow$ E-Step

# Where to use generative models?

- Generative models are useful for **data augmentation** because they can generate new examples that are similar to the training data. This can be useful when the training data is limited or when we want to increase the size of the dataset to improve the performance of a machine learning model.

- Generative models can also be used to **synthesize images and videos**, either from random noise or by sampling the model's distribution based on a given input. This can be used to generate realistic-looking images or videos, or to manipulate existing images and videos in various ways.

- In the field of **natural language processing**, generative models can be used for language translation, where the model is trained to translate text from one language to another. They can also be used for text generation, where the model is trained to generate text that is similar to a given input.

- Generative models can also be used for **anomaly detection**, where the model is trained to identify patterns in normal data and flag any data points that do not conform to these patterns as anomalous. This can be useful for identifying fraudulent activity, for example.

# End